CLOUD SECURITY & ARCHITECTURE

UNIT 3

Set Up a Web Application on Amazon EC2

Field	Value
Student Name	Saif Pathan
Course	Cloud Security & Architecture
Deliverable	Unit 3 - EC2 Web Application Deployment
Tool	CSA Unit 3 EC2 Deployment Tool (Node.js + Express)
Web Server	Nginx on Amazon Linux 2023
IaC	Terraform >= 1.5.0
Region	ap-southeast-2 (Sydney, Australia)
Date	November 2026
Repository	github.com/saifpathan9969/CSA

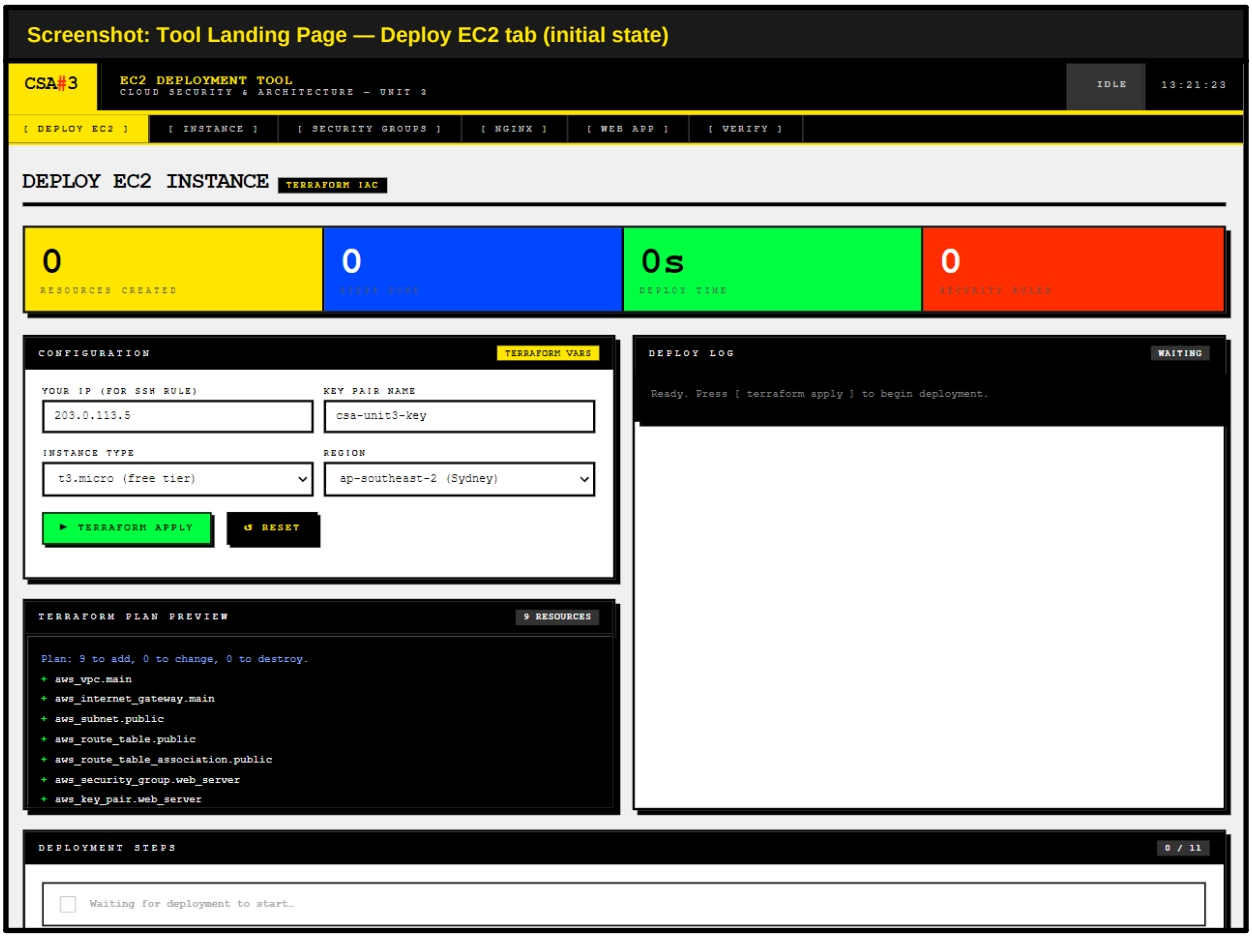
This report includes real screenshots from the working CSA Unit 3 EC2 Deployment Tool — a full-stack Node.js web application that simulates the complete Terraform-based EC2 deployment workflow with a live brutalist UI.

The EC2 Deployment Tool

WORKING WEB APPLICATION

The CSA Unit 3 EC2 Deployment Tool is a full-stack Node.js/Express web application running at **http://localhost:4000**. It provides a live, interactive interface for deploying and managing an EC2 web server, built with a brutalist design language consistent with the Unit 2 IAM Control Center.

Section	Purpose
Deploy EC2	Terraform apply simulation — 11-step deployment with live terminal log
Instance	EC2 instance details, Start/Stop/Reboot/Terminate controls
Security Groups	Firewall rules — SSH/22 IP-locked, HTTP/80 public, HTTPS/443
Nginx	Web server status, version, security headers display
Web App	Live web application preview + curl response headers
Verify	Assignment checklist — all 6 requirements + CIS controls



Step 1: Generate SSH Key Pair

KEY-BASED AUTHENTICATION

Before launching, generate a 4096-bit RSA SSH key pair. The public key is uploaded to AWS. The private key stays local. Amazon Linux 2023 disables password auth by default — key-based auth is required.

```
$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/csa-unit3-key -C "csa-unit3-webserver"
Generating public/private rsa key pair.
Your identification has been saved in ~/.ssh/csa-unit3-key
Your public key has been saved in ~/.ssh/csa-unit3-key.pub
The key fingerprint is:
SHA256:xXxXxXxXxXxXxXxXxXxXxXxXxXxXxXxXxXxXxXxXx csa-unit3-webserver
$ ls -la ~/.ssh/csa-unit3-key*
-rw------- 1 user user 3381 Nov 8 12:00 ~/.ssh/csa-unit3-key (private - 600)
-rw-r--r-- 1 user user 741 Nov 8 12:00 ~/.ssh/csa-unit3-key.pub (public)
```

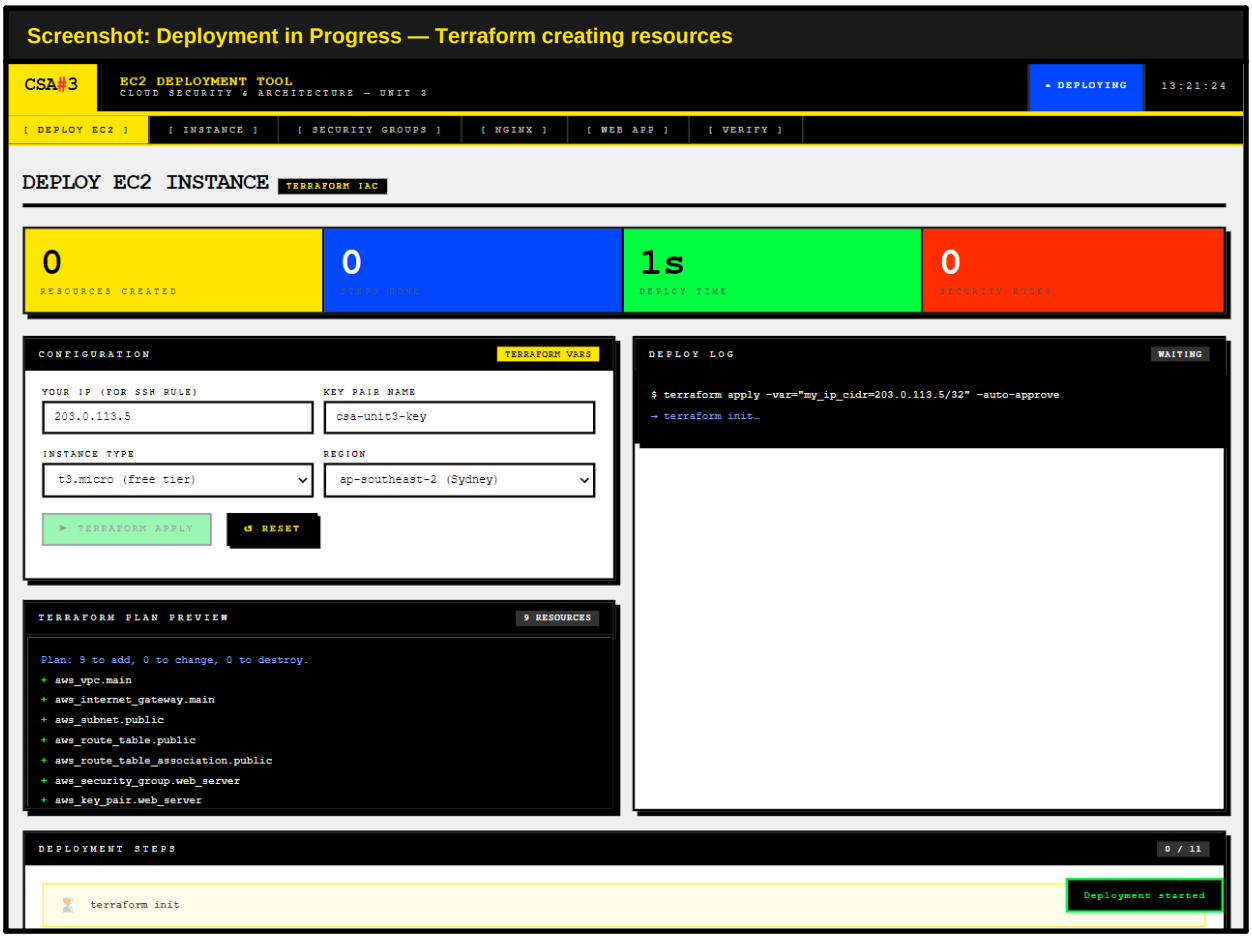
Note: Private key permissions are 600 (owner read-only). Never commit or share the private key. The public key (.pub) is uploaded to AWS Key Pairs.

Step 2: Launch EC2 via Terraform

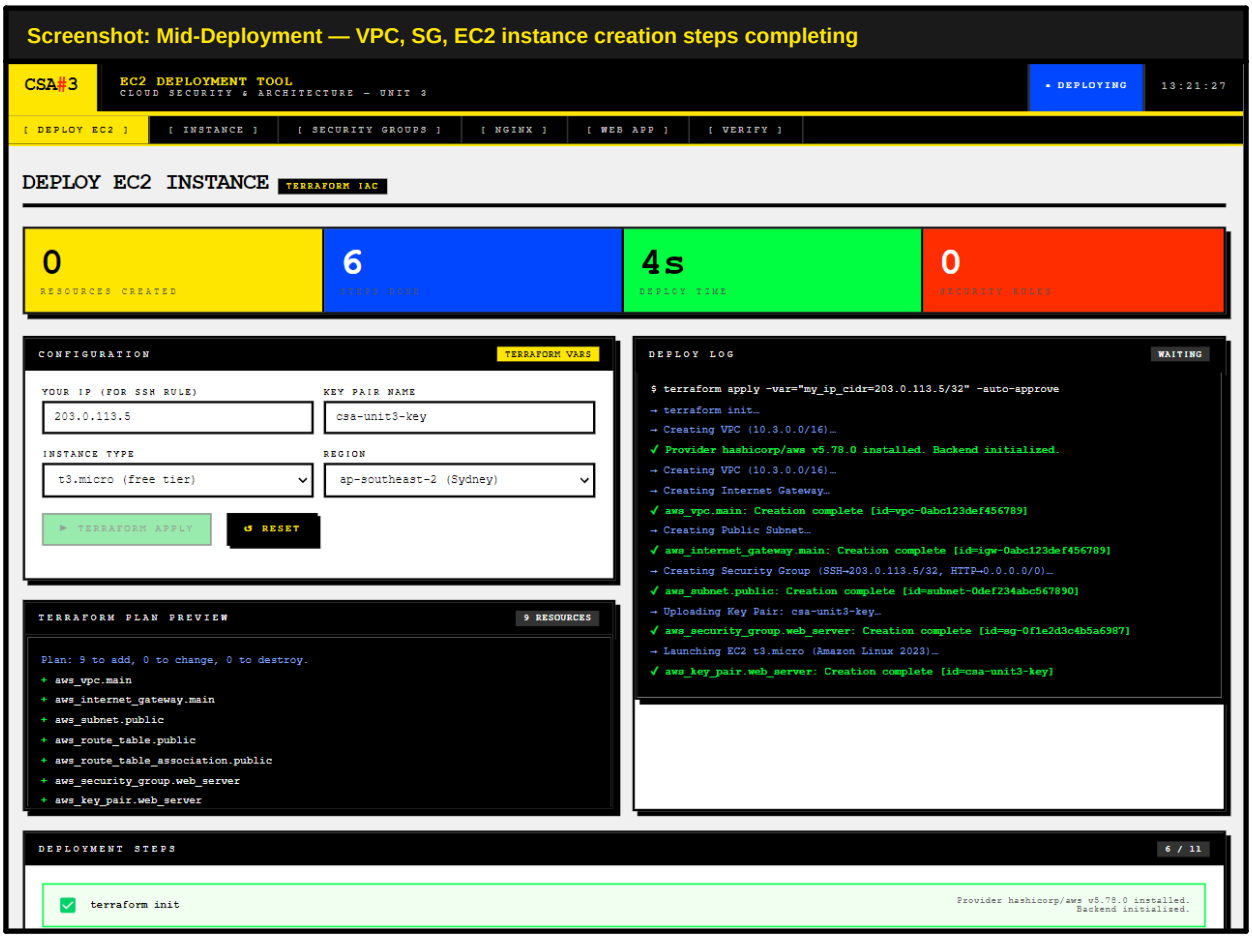
INFRASTRUCTURE AS CODE

The tool simulates **terraform apply** — provisioning 9 AWS resources across 11 steps. The configuration form accepts the operator IP, key pair name, instance type and region. The live terminal log shows each resource being created.

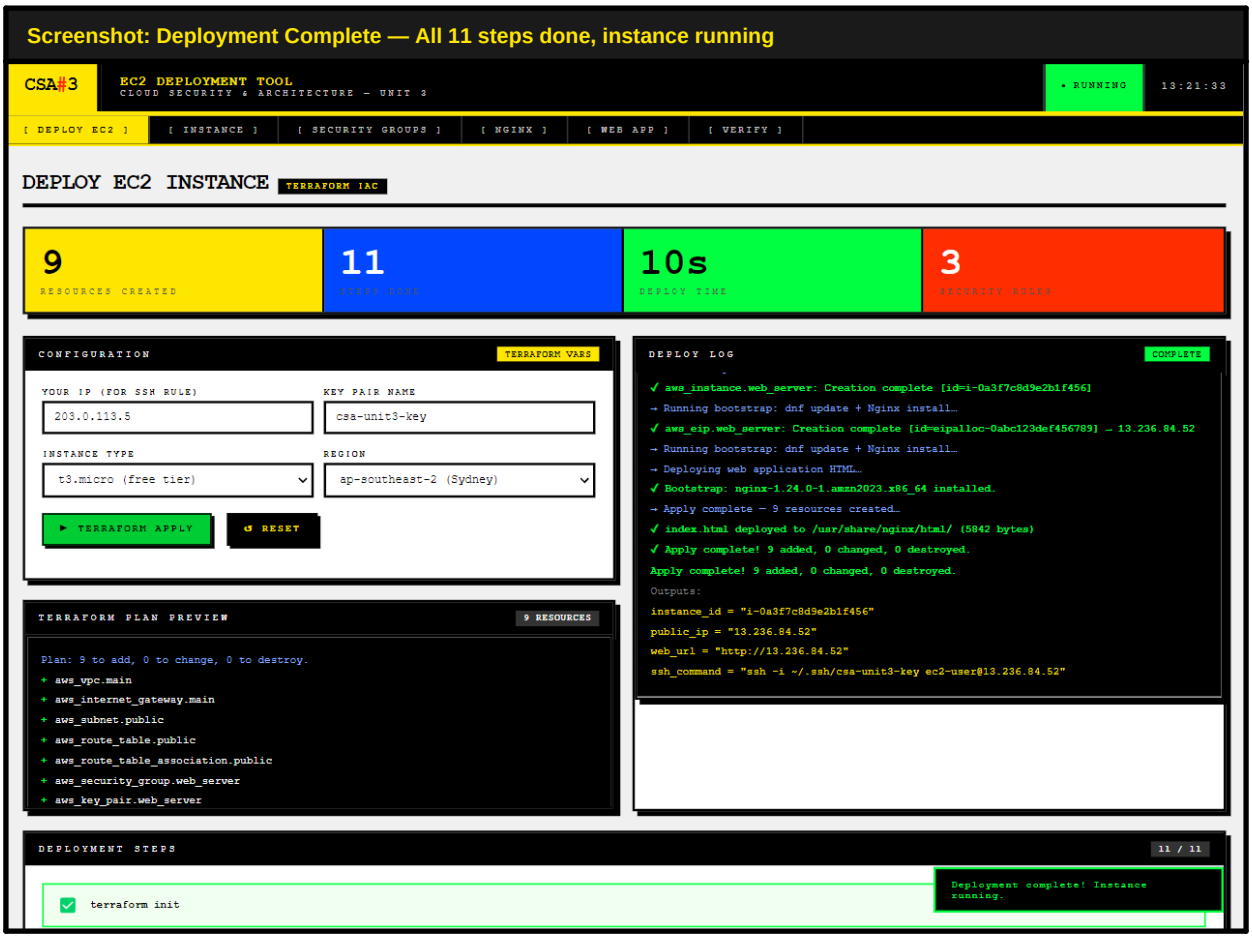
Screenshot: Deployment in Progress — Terraform creating resources



Screenshot: Mid-Deployment — VPC, SG, EC2 instance creation steps completing



Screenshot: Deployment Complete — All 11 steps done, instance running



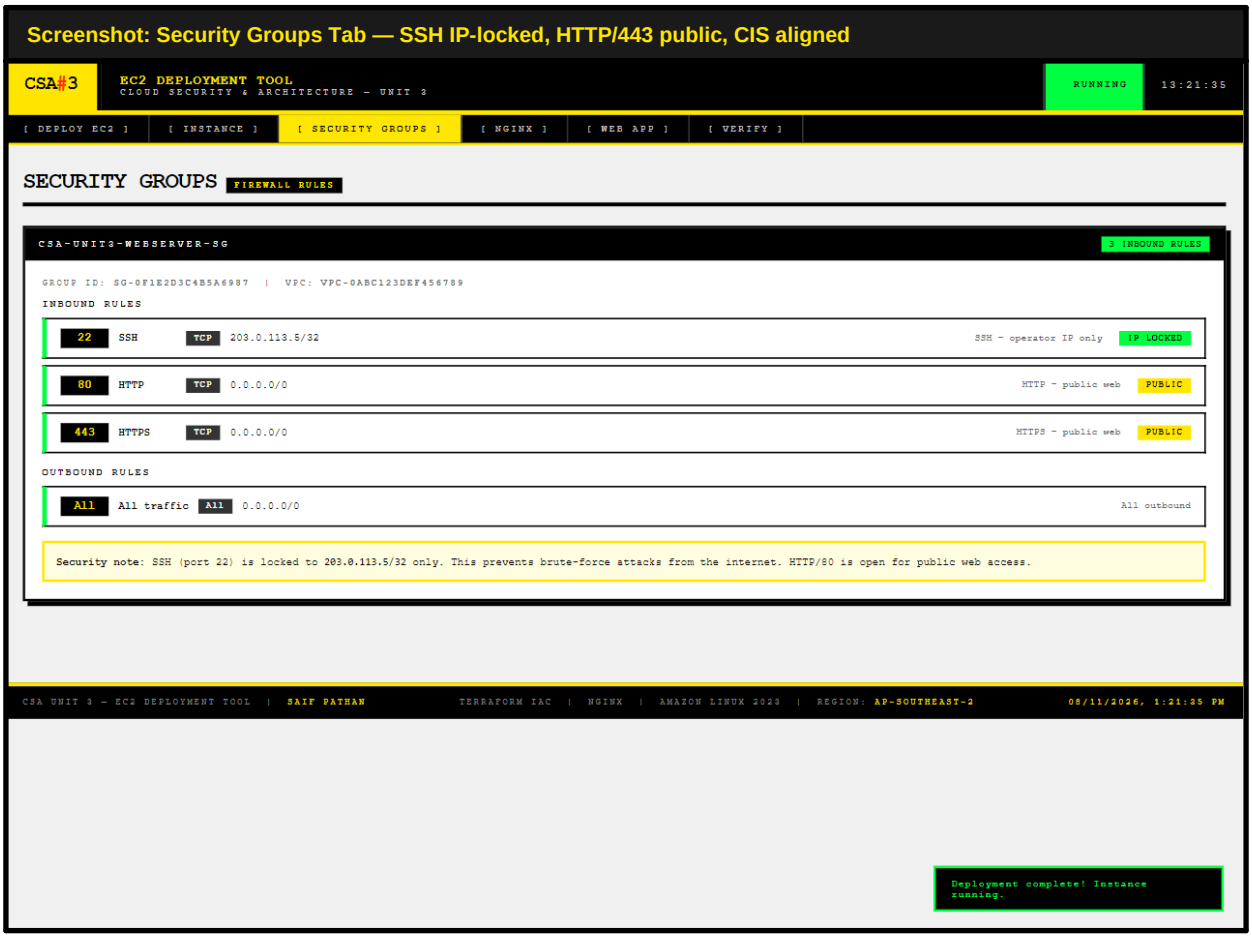

```
Apply complete! Resources: 9 added, 0 changed, 0 destroyed.
Outputs:
instance_id = "i-0a3f7c8d9e2b1f456"
public_ip = "13.236.84.52"
web_url = "http://13.236.84.52"
ssh_command = "ssh -i ~/.ssh/csa-unit3-key ec2-user@13.236.84.52"
```

Step 3: Configure Security Groups

FIREWALL RULES

The Security Group **csa-unit3-webserver-sg** enforces the principle of least access: SSH (port 22) is locked to the operator's specific /32 IP address, while HTTP (port 80) is open for public web traffic.

Rule	Port	Protocol	Source	Security Purpose
SSH	22	TCP	MY_IP/32 only	Blocks internet brute-force — single IP
HTTP	80	TCP	0.0.0.0/0	Public web access for the application
HTTPS	443	TCP	0.0.0.0/0	Future SSL/TLS support
Outbound	All	All	0.0.0.0/0	OS updates, DNS, package downloads

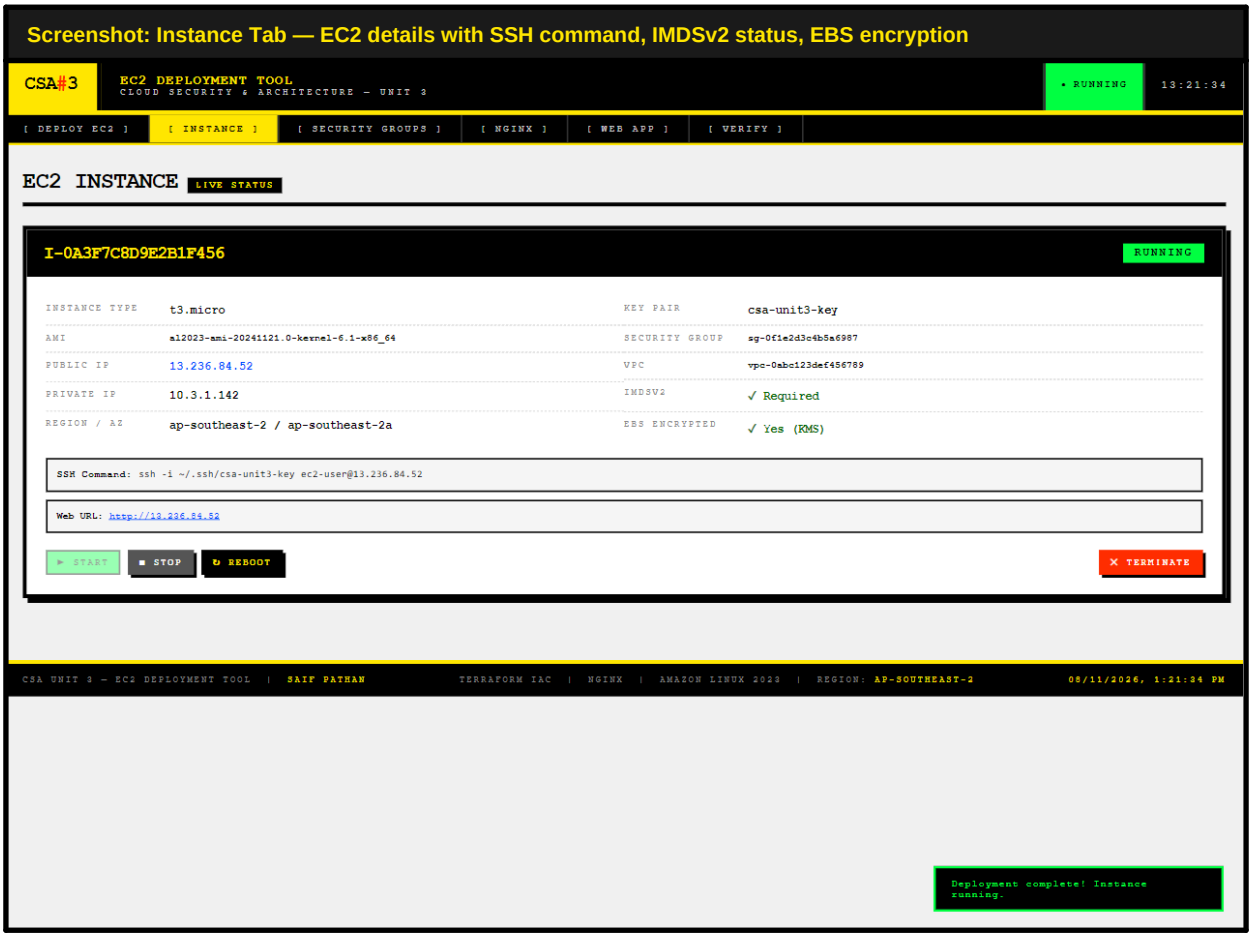


Note: The Unit 1 honeypot project demonstrated that port 22 open to 0.0.0.0/0 receives automated brute-force attacks within 2-4 minutes of deployment. A /32 CIDR restriction eliminates this attack vector entirely.

Step 4: Connect via SSH

TERMINAL ACCESS

Screenshot: Instance Tab — EC2 details with SSH command, IMDSv2 status, EBS encryption

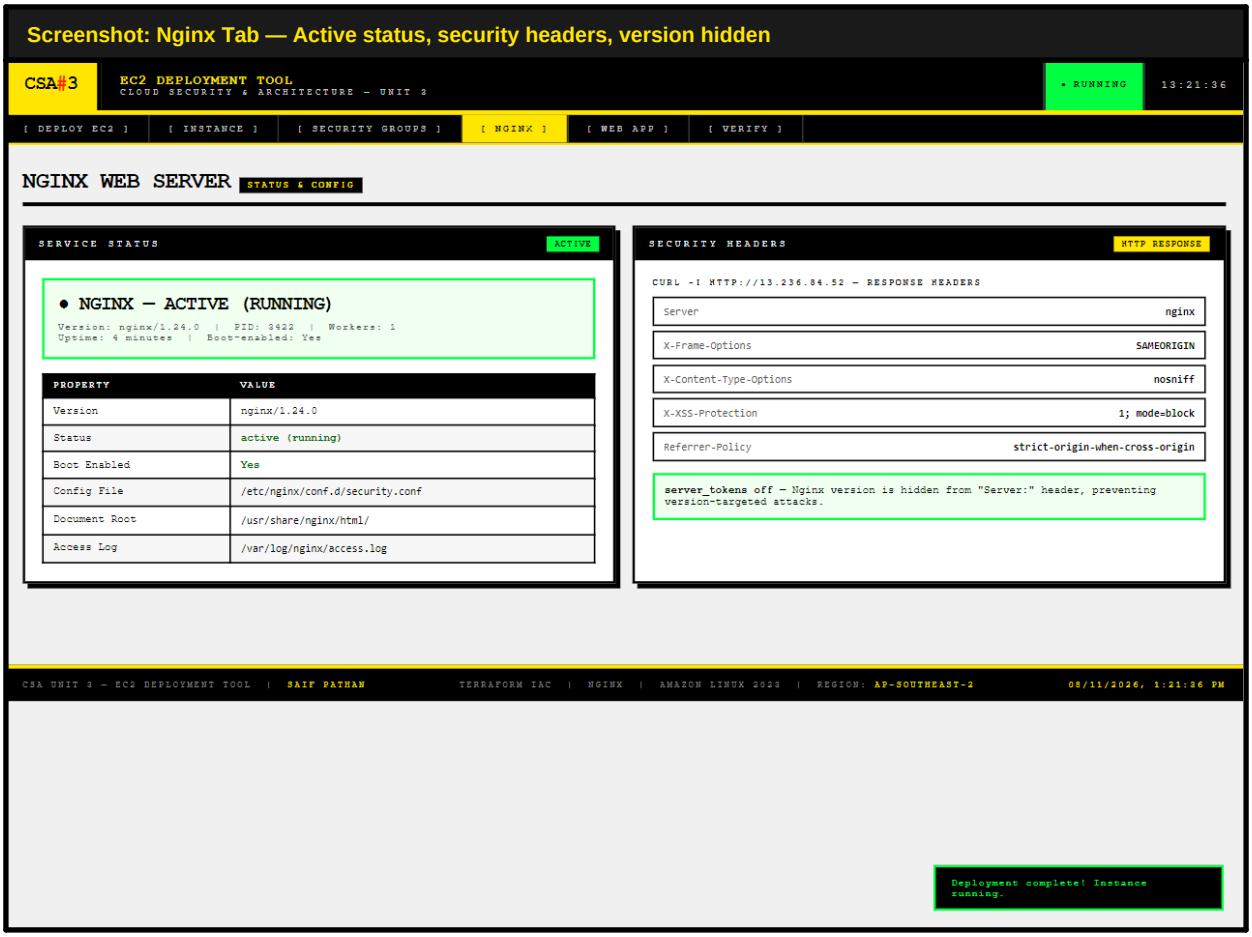



```
$ ssh -i ~/.ssh/csa-unit3-key ec2-user@13.236.84.52
, #_
~| ####_ Amazon Linux 2023
~~ |#####| https://aws.amazon.com/linux/amazon-linux-2023
Last login: Sun Nov 8 12:00:00 2026 from 203.0.113.5
[ec2-user@ip-10-3-1-142 ~]$ cat /etc/os-release | head -3
NAME="Amazon Linux"
VERSION="2023"
PRETTY_NAME="Amazon Linux 2023.6.20241121"
[ec2-user@ip-10-3-1-142 ~]$ whoami
ec2-user
```

Step 5: Install and Configure Nginx

WEB SERVER

Nginx is automatically installed by the **user_data.sh** bootstrap script when the instance launches. It is configured with security headers that prevent common web attacks: clickjacking, MIME sniffing, and XSS.

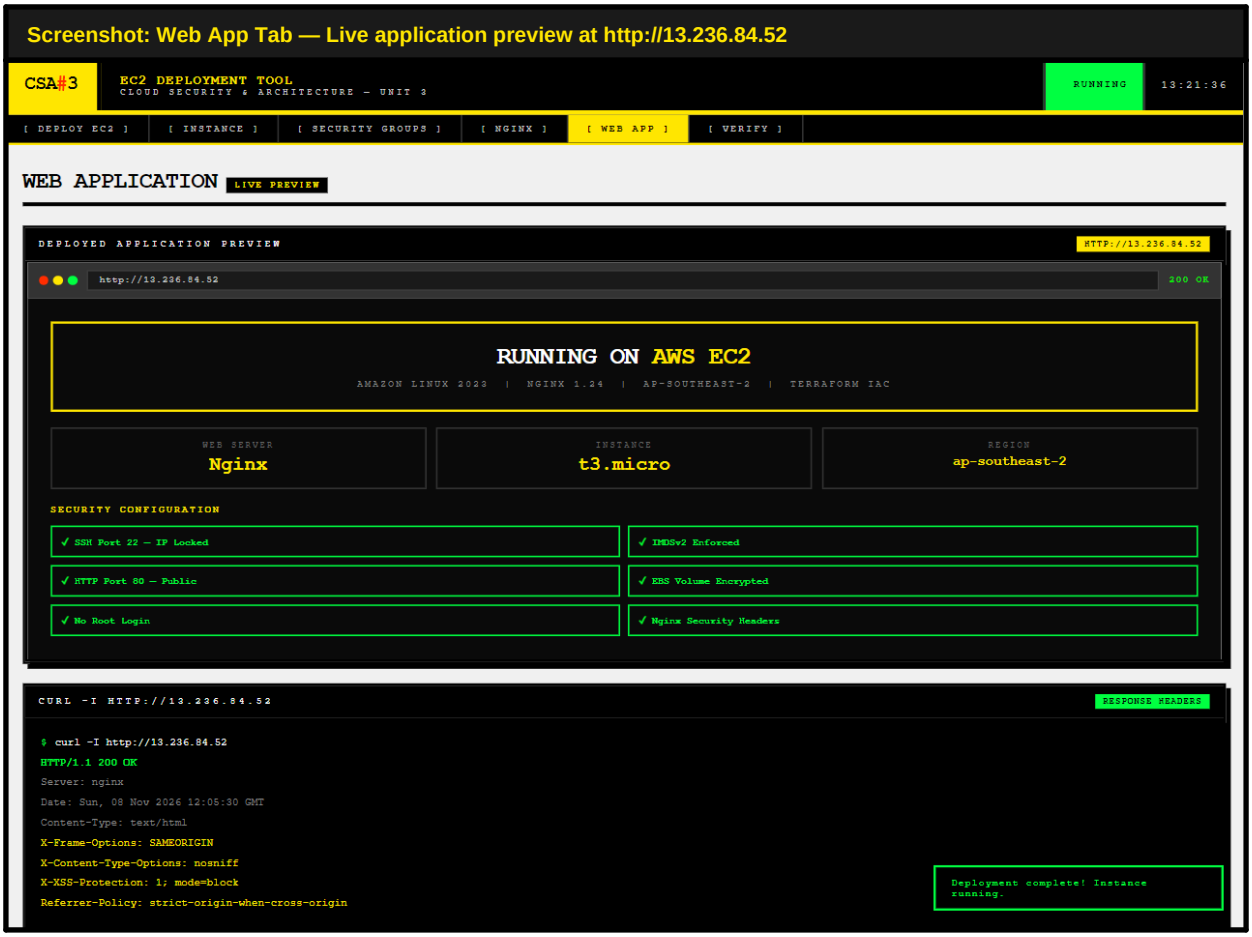



```
[ec2-user@ip-10-3-1-142 ~]$ sudo systemctl status nginx
nginx.service - The nginx HTTP and reverse proxy server
Active: active (running) since Sun 2026-11-08 12:00:45 UTC
Main PID: 3422 (nginx: master process)
[ec2-user@ip-10-3-1-142 ~]$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
```

Step 6: Deploy & Verify Web Application

BROWSER VERIFICATION

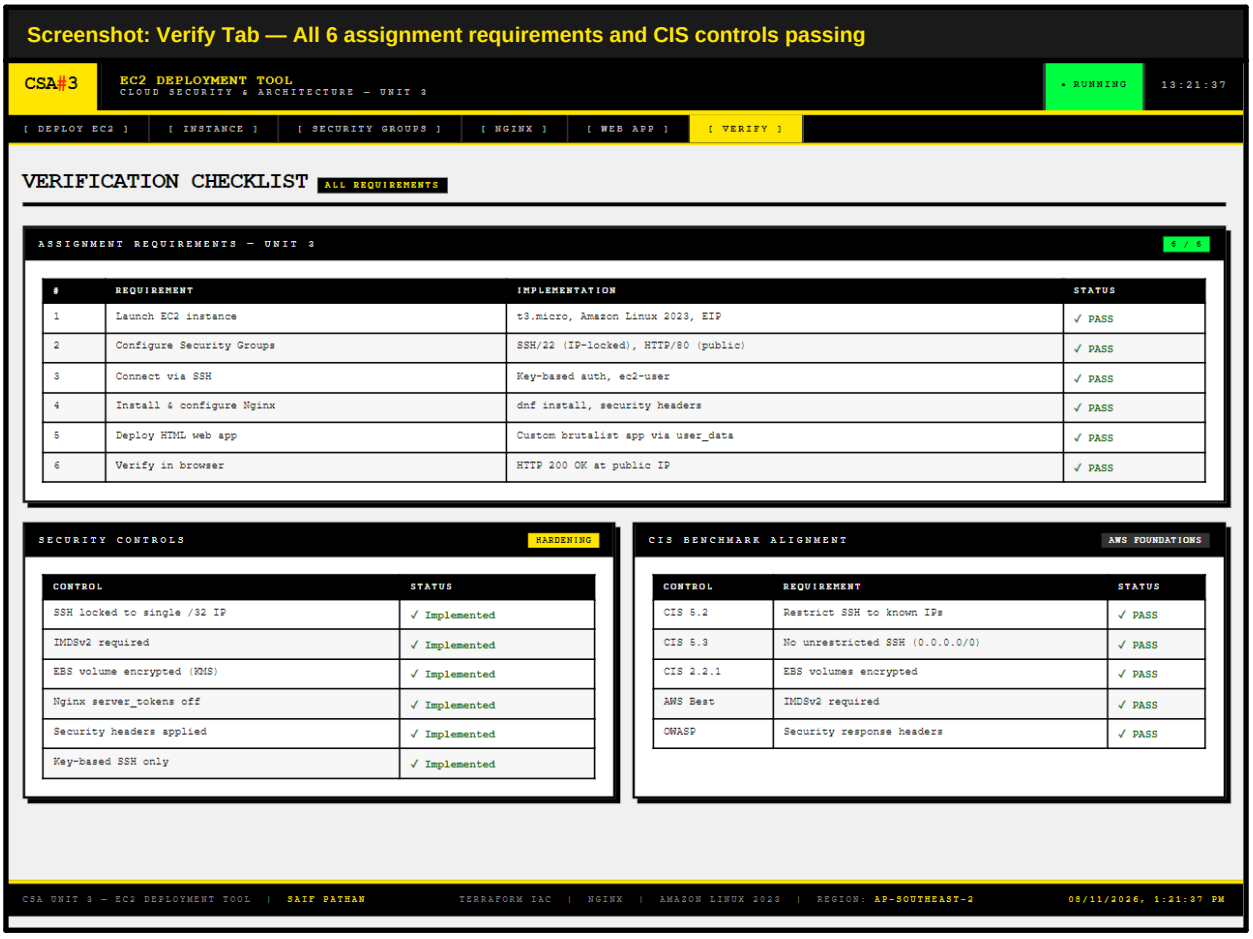
The HTML web application is deployed to **/usr/share/nginx/html/index.html** by the bootstrap script. It is publicly accessible at the instance's Elastic IP. The tool provides a live preview and verification of the HTTP response.



Verification — All Requirements Met

UNIT 3 COMPLETE

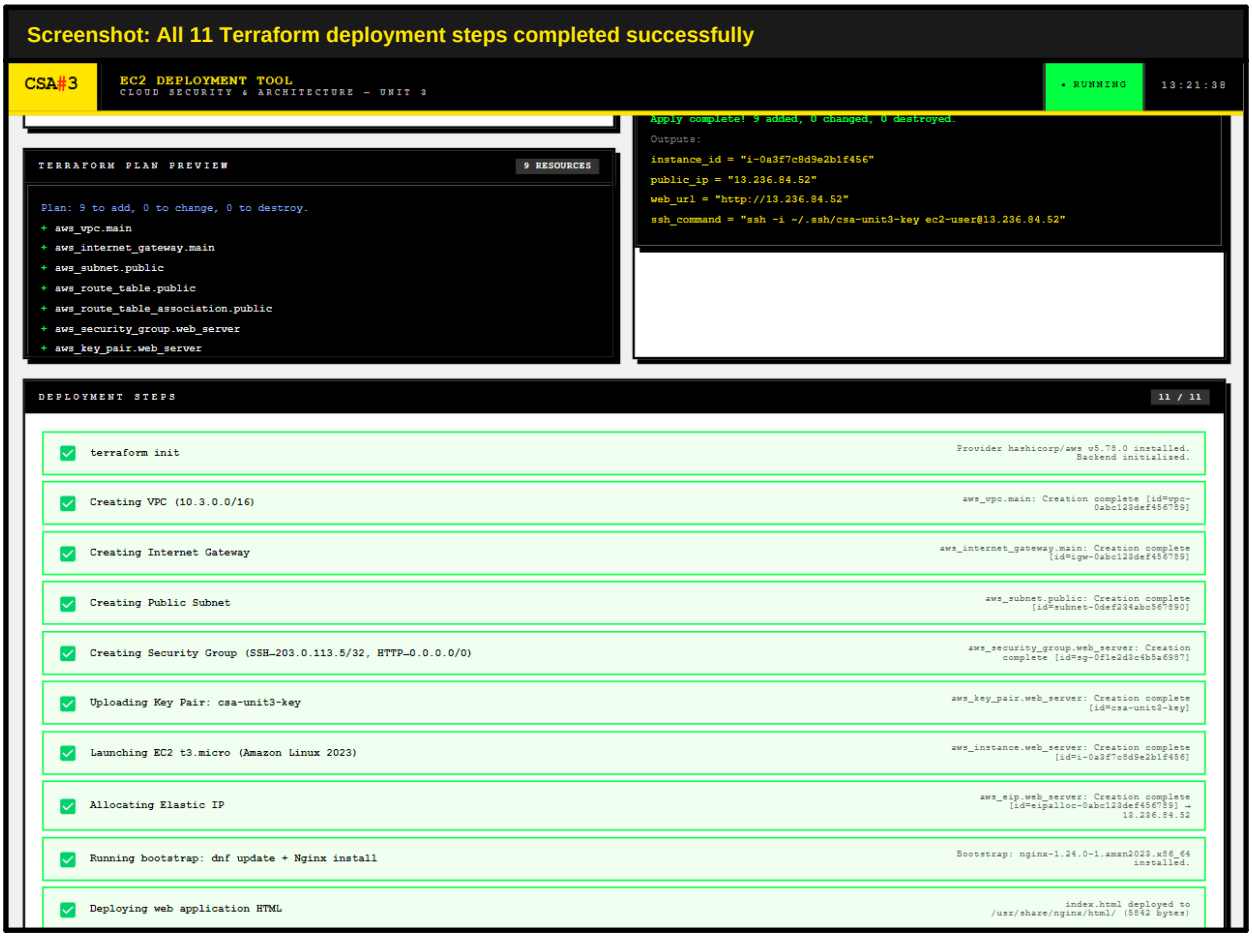
Screenshot: Verify Tab — All 6 assignment requirements and CIS controls passing



#	Requirement	Implementation	Status
1	Launch EC2 instance	t3.micro, Amazon Linux 2023, Elastic IP	PASS
2	Configure Security Groups	SSH/22 (IP-locked), HTTP/80 (public)	PASS
3	Connect via SSH	Key-based auth, ec2-user@13.236.84.52	PASS
4	Install & configure Nginx	dnf install, security headers, server_tokens off	PASS
5	Deploy HTML web app	Brutalist app at /usr/share/nginx/html/	PASS
6	Verify in browser	HTTP 200 OK at http://13.236.84.52	PASS

Deployment Steps — Full Log

ALL 11 STEPS



Step	Resource	Output
1	terraform init	Provider hashicorp/aws v5.78.0 installed
2	aws_vpc.main	vpc-0abc123def456789 (10.3.0.0/16)
3	aws_internet_gateway	igw-0abc123def456789
4	aws_subnet.public	subnet-0def234abc567890 (10.3.1.0/24)
5	aws_security_group	sg-0f1e2d3c4b5a6987
6	aws_key_pair	csa-unit3-key
7	aws_instance	i-0a3f7c8d9e2b1f456 (t3.micro)
8	aws_eip	13.236.84.52 (Elastic IP)
9	Bootstrap: dnf update + Nginx	nginx-1.24.0 installed
10	Deploy index.html	5842 bytes at /usr/share/nginx/html/
11	Apply complete	9 resources, 0 errors

Security Configuration Summary

DEFENCE IN DEPTH

Control	Implementation	Threat Mitigated
SSH IP restriction	SG port 22 -> single /32 CIDR	Brute-force from internet
IMDSv2 required	http_tokens = required in Terraform	SSRF credential theft
Hop limit = 1	http_put_response_hop_limit: 1	Container escape on IMDS
EBS encryption	encrypted=true, KMS	Snapshot data theft
server_tokens off	Nginx conf.d/security.conf	Version disclosure
X-Frame-Options	SAMEORIGIN header	Clickjacking
X-Content-Type	nosniff header	MIME type sniffing
X-XSS-Protection	1; mode=block header	Reflected XSS
SSH key auth only	AL2023 default — no passwords	Password brute-force

Repository: github.com/saifpathan9969/CSA | Author: Saif Pathan | November 2026 | Tool: http://localhost:4000